mosaickit 软件包：二维图形组装工具

Pin Yue Sung[*†]　　Ling Tak Douglas Chung[*]

2026-10-03 · v0.5.1 · ·

目录

1 简介 2
1.1 设计 2
1.2 数学与证明 2
1.3 阅读指引 2
2 安装 3
2.1 系统需求 3
2.2 安装软件包 3
2.3 开发环境 3
3 快速入门 4
4 画布与场景 5
4.1 画布规格 5
4.2 画布 5
4.3 场景 6
4.4 错误与警告 6
5 图层与坐标轴 6
5.1 共同字段 6
5.2 基本图层 7
5.3 坐标轴 8
6 坐标轴注释 8
6.1 标记、注释与大括号 8
6.2 边栏的字段 9
6.3 沿轴分散文字 9
6.4 大括号的轨道 10
7 布局几何 11
7.1 矩形 11
7.2 方向与线段 12
7.3 多边形 12
7.4 区域的视觉中心 13
7.5 大括号外形 14
8 区域标签与点标签 15
8.1 区域标签 15
8.2 点标签 15
8.3 硬性限制 16
8.4 引线标注 17
9 样式与颜色 18
9.1 稀疏样式 18
9.2 样式类型 19
9.3 颜色 20
9.4 调色板 20
10 主题与配置 21
10.1 样式组与主题 21
10.2 角色解析 22
10.3 注册领域角色 23
10.4 配置 23
11 参数、网格与动画 24
11.1 参数与表达式 24
11.2 网格 26
11.3 动画 27
12 绘制 27
12.1 渲染器 27
12.2 Matplotlib 渲染器 27
12.3 结果与保存 28
12.4 缓存 28
A 证明 28
A.1 排列与轨道 29
A.2 矩形、线段与多边形 30
A.3 不可及极点 31
A.4 标签 33
A.5 样式、主题与绑定 33
更新记录 35
命令索引 37
参考文献 39

[*]台湾政治大学国际经营与贸易学系，台北，台湾。
[†]通讯作者。电子邮箱：contact@econ-viz.org

第 1 节　简介

mosaickit 软件包以小型且不可变的组件组装二维图形：场景是一份有序的图层列表（路径、填色区域、标记、文字、箭头、标签、大括号）；每个图层标明一个语义角色，主题将角色转换为样式，渲染器再将结果输出为 PNG、SVG、PDF、GIF 或 MP4。它不包含任何特定领域的知识。principle-viz、utility-viz 等领域软件包自行定义模型与角色名称，再将要绘制的图层交给 mosaickit；曲线构造与 TikZ 导出则由 bezierkit 等几何软件包负责。

1.1　设计

软件包设计遵循四项原则。

不可变的值　图层、场景、样式、主题与规格都是冻结的 dataclass。`Canvas` 是封装不可变 `Scene` 的流式构建器；`snapshot()`、`copy()` 与 `bind()` 不会改动其他对象持有的场景。

稀疏样式　每个样式字段都可以是 `None`，表示继承。图层自带的样式位于画布覆盖、配置覆盖、主题与基本默认值之上（详见第 10 节）。

角色而非颜色　图层只说明自己是什么（`"primary"`、`"axes.note"`、`"mypkg.boundary"`），外观由主题决定，而颜色是画布绘制时才解析的调色板名称（详见第 9 节）。

避让所有内容的布局　区域标签、点标签、大括号与坐标轴注释会等其他内容绘制完毕后再放置。布局仅依据显示像素进行几何计算，使文字不碰到任何线、标记、区域或其他文字（详见第 7 节、第 8 节）。

1.2　数学与证明

自动布局以多种计算几何方法为基础。方向测试用于判断点位于有向直线的哪一侧，奇偶规则则依据射线穿越边界的次数判断内外；此外还会计算到多边形边界的距离，并以每次优先检查上界最高候选区域的最佳优先搜索找出区域最深处。坐标轴文字则会排列到互不重叠且位移平方和最小的位置。各章以编号的定义、引理、命题与定理说明每个程序的保证，证明集中在附录 A；如果只想查阅 API，可以略过。样式与主题的代数（稀疏合并、角色解析）以及参数绑定也采用相同的处理方式。几何部分的标准参考书为 de Berg et al. (2008)，定理 2 背后的保序最小二乘问题则参见 Barlow et al. (1972)。

1.3　阅读指引

表 1　章节指引

主题	内容	章节
构造	画布、规格、场景	第 4 节
	路径、填色、标记、文字、坐标轴	第 5 节
	坐标轴标记、注释与大括号	第 6 节
布局	布局几何	第 7 节

主题	内容	章节
	区域标签与点标签	第 8 节
外观	样式、颜色、调色板	第 9 节
	主题、角色、配置	第 10 节
输出	参数、网格、动画	第 11 节
	渲染器、缓存、保存	第 12 节

初次使用请先读第 3 节、第 4 节与第 5 节。本手册的图都是 mosaickit 自己的输出：每张图由它所示范的画布或网格绘制，并以印在此处的尺寸存成 PDF。

绘制本手册各图的脚本按主题分类，可一次下载：mosaickit-examples.zip。

第 2 节　安装

2.1　系统需求

mosaickit 需要 Python 3.10 以上、NumPy 1.24 以上与 Matplotlib 3.6 以上（4 以下）；在 Python 3.10 上还会安装 tomli 来读取 TOML。GIF 输出使用 Pillow，Matplotlib 本身已依赖此软件包；MP4 输出则需要 `PATH` 中有 `ffmpeg`。

2.2　安装软件包

```
uv add mosaickit                  # the library
uv add "mosaickit==0.5.1"         # the version this manual describes
```

如果使用 pip，请运行 `python -m pip install mosaickit`。导入 `mosaickit` 时不会一并导入 Matplotlib：内置渲染器要到画布第一次绘制时，才会按名称加载（详见第 12 节）。

2.3　开发环境

```
git clone https://github.com/EconViz/mosaickit.git
cd mosaickit
uv sync --locked
uv run pre-commit install
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy
uv run lint-imports
uv build
```

锁文件固定了所有开发依赖包的版本。持续集成流程会在 Python 3.10、3.11、3.12 与 3.13 上运行相同命令，再将构建完成的 wheel 安装到干净环境，确认其中不含 bezierkit，最后以该 wheel 运行测试。导入规则维持分层：场景、样式、主题与参数模块不会导入绘制与画布模块，核心也不会导入任何领域软件包。

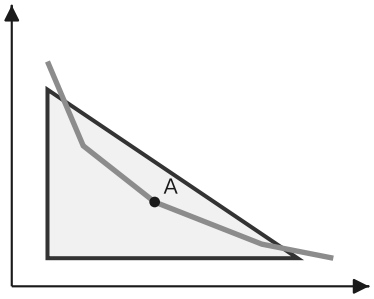


图 1
快速入门的图

第 3 节　快速入门

一张图由画布、一组图层与一次保存构成：

例 1

```
from mosaickit import (
    Canvas, Fill, FillLayer, MarkerLayer, PathLayer, Stroke, TextLayer,
    quadrant_axes,
)

canvas = Canvas().extend(quadrant_axes(10, 10))
canvas.add(
    FillLayer(
        [(1, 1), (1, 7), (8, 1)],
        fill=Fill(color="#377EB8", opacity=0.12),
        z_index=-1,
    )
)
canvas.add(
    PathLayer(
        [(1, 8), (2, 5), (4, 3), (7, 1.5), (9, 1)],
        stroke=Stroke(color="#984EA3", width=2),
        id="curve",
    )
)
canvas.add(MarkerLayer([(4, 3)], id="point"))
canvas.add(TextLayer((4, 3), "A", offset=(6, 6)))
canvas.save("diagram.pdf")
```

`quadrant_axes(10, 10)` 返回一般图层：两条带箭头的路径及其标题。填色图层以 `z_index=-1` 放在最底层。路径是通过各点的折线；mosaickit 本身不直接绘制曲线。如果要呈现平滑曲线，必须传入大量采样点，或先用几何软件包构造曲线再采样。文字从该点向右上偏移 6 pt。调用 `save()` 之前不会进行任何绘制；`save()` 会用画布的渲染器（未另行设置时为 Matplotlib）绘制场景、写出文件，并返回输出路径。

同一个画布可在修改后再次保存：`add()`、`extend()`、`remove()` 与 `clear()` 都会返回画布本身，因此可以链式调用；先前获取的快照则维持原状（详见第 4 节）。能自动避让其他内容的标签也是图层：将 `TextLayer` 换成 `PointLabelLayer((4, 3), "A")`，渲染器就会自行选择放置方向（详见第 8 节）。

第 4 节　画布与场景

4.1　画布规格

CanvasSpec `CanvasSpec(x_range=(0, 10), y_range=(0, 10), width=6.0, height=6.0, dpi=300, x_label="X", y_label="Y", title=None)`
图形的物理尺寸与坐标范围。`x_range` 与 `y_range` 是显示的数据范围；`width` 与 `height` 是以英寸计算的图形尺寸；`dpi` 是 $[1, 1200]$ 内的整数。属性 `x_min`、`x_max`、`y_min`、`y_max` 可读取范围，`replace(**changes)` 则返回只修改指定字段的副本。范围必须有限且 `lo < hi`，尺寸必须是有限正数，否则会引发 `ConfigurationError`。

Interval `Interval(⟨lo⟩, ⟨hi⟩)`
有限的递增区间，`lo < hi`，供规格与坐标轴共用。

4.2　画布

Canvas `Canvas(spec=None, theme=None, config=None, renderer=None, *, role_overrides=None)`
封装不可变场景的流式构建器。如果参数为 `None`，便采用创建画布当下生效的配置（详见第 10.4 节），分别是其中的 `canvas_spec`、`theme` 与 `renderer`。`role_overrides` 将角色名称映射到 `StyleBundle`，并应用在主题与配置之上（参见定理 30）。

`add(layer)`, `extend(layers)` 加入一个或多个图层；返回画布。

`remove(layer_id)`, `clear()` 移除该 id 的图层（含群组内）或所有图层；返回画布。未知的 id 引发 `ConfigurationError`。

`snapshot()` 目前的 `Scene`。之后调用 `add()` 不会改变它。

`copy()` 规格、主题、配置、渲染器、覆盖、场景与绑定都相同的新画布。

`bind(parameter, value)`, `bind(mapping)` 新画布，其场景中的这些参数换成数值（详见第 11 节）；原画布保留参数。

`render(*, renderer=None, cache=None)` 绘制场景并返回渲染器的结果（详见第 12 节）。

`save(target, *, renderer=None, cache=None, **options)` 绘制、写出 `.png`、`.pdf` 或 `.svg`、关闭结果并返回写出的路径。选项即 `SaveOptions` 的字段。

构造方法会修改画布，但不会就地改动场景：每次调用都以新场景取代画布中的场景，因此先前获取的快照、副本或绑定后的画布都会维持原有内容。

```
from mosaickit import Canvas, PathLayer

canvas = Canvas()
before = canvas.snapshot()
canvas.add(PathLayer([(0, 0), (1, 1)], id="line"))
assert before.layers == ()              # the old snapshot is unchanged
assert canvas.snapshot().layers[0].id == "line"
```

4.3　场景

Scene　`Scene(layers=(), metadata={})`

具有持久性且有序的图层集合，也就是每次操作都会保留旧版本。`add()`、`extend()`、`remove()` 与 `clear()` 都会返回新场景；`Scene.empty()` 是空场景。图层 id 在整个场景（包括群组）中必须唯一，如有重复便引发 `ConfigurationError`。`ordered_layers` 按 `z_index` 排序顶层图层；值相同时维持加入顺序，而这个顺序就是绘制顺序。

4.4　错误与警告

表 2
异常与警告

类	引发或发出的时机
`MosaicKitError`	以下三个错误的基础类
`ConfigurationError`	模型、样式、主题、规格或配置值无效
`BindingError`	参数缺少值、类型错误，或表达式无法求值
`RenderError`	渲染器无法完成要求：未知的格式或渲染器、缺少的图例项目
`LayoutWarning`	自动布局无法避开所有障碍物（详见第 8 节）
`CacheBypassWarning`	图层的模型无法哈希，因此不经缓存绘制

第 5 节　图层与坐标轴

5.1　共同字段

Layer　`Layer(*, id=`⟨*uuid*⟩`, role="primary", z_index=0, visible=True, legend=None, model=None)`

所有图层共用的不可变基类。各种图层在自身的位置参数之后，都接受以下仅限关键字使用的字段。

id　非空字符串，在场景中唯一；默认为随机十六进制字符串。图例、区域标签与 `remove()` 都通过 id 指定图层。

role　以点分隔的角色名称，例如 `"primary"` 或 `"mypkg.boundary"`；它决定图层从主题获取的样式（详见第 10 节）。每种图层都有自己的默认角色，以及最后才会查询的回退类别。

z_index　有限的绘制顺序；数值越大越晚画、越在上层。群组会把自己的 `z_index` 加到子图层上。

visible　`False` 时绘制会略过该图层。

legend　图层在图例中的标签；没有标签的图层不列入图例。

model　用于创建该图层的任意领域对象。它会参与绘制缓存（详见第 12 节），但不参与相等比较与绑定。

坐标可以是有限实数对（数据单位），也可以是参数表达式（详见第 11 节）。无效的几何会在创建图层时引发 `ConfigurationError`，而不是等到绘制时才报错。

5.2　基本图层

表 3　基本图层

图层	默认角色	绘制内容
`PathLayer(path, stroke)`	`primary`	通过至少两点的折线
`FillLayer(boundary, fill, stroke)`	`region`	至少三点的多边形，含填色与外框
`MarkerLayer(points, marker)`	`point`	每个点上的标记，至少一点
`TextLayer(position, text)`	`text`	某点上的文字
`ArrowLayer(start, end, stroke)`	`annotation`	直线箭头，可加标签
`LegendLayer(entries, style)`	`legend`	有标签图层的图例
`GroupLayer(children)`	`primary`	本身不画任何东西，只将图层分组

`PathLayer` `PathLayer(path, stroke=None, arrow_placement=ArrowPlacement.END, clip=True, *, ...)`
以直线段连接各点。如果解析后的线条样式设有 `arrow`，便会在 `START`、`END` 或 `BOTH` 端画出与该端线段对齐的箭头。`clip=False` 可让线条以完整宽度越过绘图区边缘，坐标轴默认便采用此设置。

`FillLayer` `FillLayer(boundary, fill=None, stroke=None, *, ...)`
以 `fill` 填色、以 `stroke` 描边的闭合多边形。`RegionLabelLayer` 以它的 id 引用它。

`MarkerLayer` `MarkerLayer(points, marker=None, *, ...)`
在每个点绘制一个标记。标记一律完整画出：中心位于绘图区边缘的标记（例如坐标轴上的点）不会被切半，中心落在绘图区外的则会略过。

`TextLayer` `TextLayer(position, text, style=None, offset=(0, 0), anchor="center", math=False, *, ...)`
在 `position` 放置文字，并偏移 `offset` pt。`anchor` 指定文字框的哪一点要对准该位置：`center`、`left`、`right`、`top`、`bottom`、`top-left`、`top-right`、`bottom-left` 或 `bottom-right`。`math=True` 时，文字会以数学式排版（`"a_1"` 成为 a_1）。

`ArrowLayer` `ArrowLayer(start, end, stroke=None, label=None, arrow_placement=ArrowPlacement.END, *, ...)`
从 `start` 指向 `end` 的直线箭头。除非线条样式指定其他 `ArrowStyle`，否则使用开放式箭头；`label` 写在中点。虚线、点线或点划线只应用在箭身，箭头维持实线。箭头不受绘图区裁切。

`LegendLayer` `LegendLayer(entries=(), style=None, *, ...)`
列出 `entries` 中各 id 所指定的图层；`entries` 为空时，则列出所有带有 `legend` 标签的图层。如果 id 不存在或对应图层没有标签，便会引发 `RenderError`。`LegendStyle(visible=False)` 可隐藏图例。

GroupLayer　`GroupLayer(children, *, ...)`
由其他图层组成的图层。子图层的绘制方式如同直接加入场景，但其 `z_index` 会再加上群组的值；隐藏群组也会隐藏其中所有子图层。

5.3　坐标轴

AxisSpec　`AxisSpec(extent, arrow=None, label=None)`

build_axes　`build_axes(x, y) -> list[Layer]`
`build_axes` 将两个坐标轴规格转换为角色为 `axes` 的一般图层。两个规格都没有 `arrow` 时，结果是围住范围的闭合框；否则，各轴会沿 $y = 0$ 或 $x = 0$ 画成直线，并按指定的 `ArrowPlacement` 画出实心三角箭头。标题位于箭头尖端之外 6 pt：x 轴标题在右侧，y 轴标题在上方。

quadrant_axes　`quadrant_axes(x_max, y_max)`

crosshair_axes　`crosshair_axes(x_range, y_range)`

box_frame　`box_frame(total_x, total_y)`
三种默认依次为：从原点出发、远端带箭头的两轴；穿过原点、两端带箭头的两轴；以及不带箭头的外框。

每条坐标轴都由 `PathLayer` 与 `TextLayer` 组成，因此可通过 `axes` 角色改变样式、以 id（`axes.x`、`axes.y`、`axes.frame`、`axes.x.label`、`axes.y.label`）移除，或改用自行创建的图层。

第 6 节　坐标轴注释

y 轴位于绘图区左缘，x 轴位于下缘，两轴外侧的空间称为*边栏*。其中三种图层放置在边栏中，第四种则在绘图区内画出大括号。

6.1　标记、注释与大括号

AxisMarkLayer　`AxisMarkLayer(axis, value, label, math=False, style=None, *, role="axes", ...)`
新增：*0.3.0*　在轴 `"x"` 或 `"y"` 的 `value` 处、绘图区外的短符号，例如 a_0 或 p^*。

AxisNoteLayer　`AxisNoteLayer(axis, value, text, style=None, *, role="axes.note", ...)`
新增：*0.3.0*　用于说明 `value` 的文字，可以分成多行，位于边栏最外侧的一栏。在默认主题中，`axes.note` 角色会让注释比标记更小、更淡（9 pt、`grey-600`）。

BraceLayer　`BraceLayer(axis, start, end, label=None, side="inside", *, role="axes", math=False, style=None, stroke=None, ...)`
新增：*0.3.0*　覆盖轴上 `start..end` 的大括号，可加上标签。`side="outside"` 会画在边栏中、标记之外；`"inside"` 则画在绘图区内侧，标签会避开所有线、点、区域与文字，尖端外没有空位时便改用引线拉出。

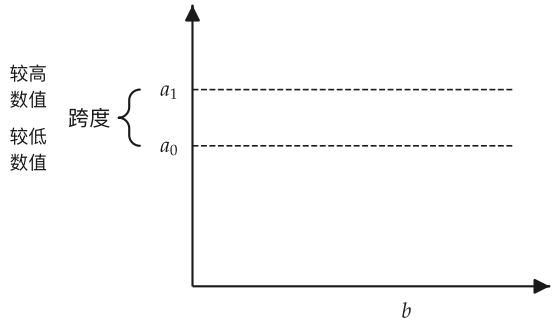


图 2
y 轴边栏的标记与注记

`SpanBraceLayer` `SpanBraceLayer(start, end, label=None, side="below", *, role="axes", math=False, style=None, stroke=None, ...)`

新增：*0.4.0*

绘图区内两点之间的大括号。跨距必须是水平（`side` 为 `"above"` 或 `"below"`）或垂直（`"left"` 或 `"right"`）；大括号会朝 `side` 凸出，与两点相距 4 pt、深 8 pt。标签位于尖端之外，并会避开其他所有内容（详见第 8 节）。

```
from mosaickit import AxisMarkLayer, AxisNoteLayer, BraceLayer, Canvas,
quadrant_axes

canvas = Canvas().extend(quadrant_axes(10, 10))
for value, symbol, note in [(7, "a_1", "Upper\nvalue"), (5, "a_0",
"Lower\nvalue")]:
    canvas.add(AxisMarkLayer("y", value, symbol, math=True))
    canvas.add(AxisNoteLayer("y", value, note))
canvas.add(BraceLayer("y", 5, 7, "Span", side="outside"))
canvas.add(AxisMarkLayer("x", 6, "b", math=True))
```

6.2　边栏的字段

各栏从轴线向外排列，第一栏与轴线相距 5 pt，各栏彼此相距 8 pt。顺序依次是标记、每条外侧大括号轨道各一栏，以及注释。每栏宽度取其中最宽的内容；空栏不占空间，也不增加间距（参见图 2，图中另加了辅助线）。随后还要处理两个问题：同一栏中相邻的文字不能重叠，以及哪些大括号可以共用一条轨道。

6.3　沿轴分散文字

同一栏中的每段文字都对应轴上的一个区间，并以其标示的值为中心。区间重叠时，系统会在维持顺序的前提下将它们分开，并使各项位移的平方和最小。

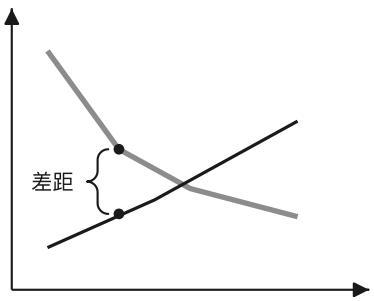


图 3
两点之间的跨距大括号

定义 1 (保序排列) 设 $c_1, \dots, c_n$ 为中心、$s_1, \dots, s_n \ge 0$ 为大小、$g \ge 0$ 为间距，编号使 $c_1 \le \cdots \le c_n$（相等者按输入顺序）。*排列*是满足

$$x_{k+1} - x_k \ge \frac{s_k + s_{k+1}}{2} + g, \quad k = 1, \dots, n-1$$

的向量 $x \in \mathbb{R}^n$；*保序排列问题*是在所有排列中最小化 $\sum_k (x_k - c_k)^2$。

spread `mosaickit.layout.stack1d.spread(centers, sizes, gap=0.0) -> tuple[float, ...]`

新增：*0.3.0* 求解此问题：将连续且重叠的项目合并成组，每组紧密排列在所有成员目标位置的平均值周围；只要某组碰到前一组，就再次合并。结果按输入顺序返回。长度不一致或大小为负时会引发 `ValueError`。

定理 2 (分散为最佳解) *spread 返回定义 1 保序排列问题的唯一解。*

这个程序采用保序回归的合并相邻违反者算法（pool-adjacent-violators algorithm, PAVA）(Ayer et al., 1955)：扣除紧密排列的位移后，间距限制就会变成 $y_1 \le \cdots \le y_n$。

推论 3 (分散结果的性质) *设 x 为 spread 的结果，则 (i) 任两项 $i \ne j$ 满足 $|x_i - x_j| \ge \frac{s_i + s_j}{2} + g$；(ii) 若各中心本身已构成排列，则 $x = c$；(iii) 每群的位移总和为零，因此群的平均位置等于其成员的平均目标。*

图 4 中的前三项因重叠而形成一组，该组的位置以各项目标位置的平均值为中心；其余两项原本就已分开，因此不会移动。

6.4　大括号的轨道

跨距（连同标签）彼此太近的大括号必须放在不同轨道，也就是与轴线不同的距离。

定义 4 (冲突区间) 对间距 $g \ge 0$，若 $b + g \le a'$ 与 $b' + g \le a$ 都不成立，则区间 $[a, b]$ 与 $[a', b']$ *冲突*。*轨道分配*为每个区间指定一个轨道编号，使冲突的区间不共用轨道。

assign_lanes `mosaickit.layout.stack1d.assign_lanes(intervals, gap=0.0) -> tuple[int, ...]`

新增：*0.3.0* 首次适配：按输入顺序处理区间，把每个区间放进与已有区间都不冲突的最低轨道。端点可按任一顺序给出。



图 4
目标与分散结果

定理 5 (首次适配使用最少轨道) *assign_lanes 一定返回轨道分配。若区间按下端递增的顺序给出，且每个都满足 $b - a + g > 0$，则它恰好使用 ω 条轨道，其中 ω 为两两冲突的区间数的最大值；没有任何轨道分配能用得更少。*

如果输入不是上述顺序，首次适配可能使用超过必要数量的轨道。因此，可能重叠的大括号应按数值从低到高加入。

gutter_columns `mosaickit.layout.gutter.gutter_columns(mark_width, brace_widths, note_width, *, start, gap)`

新增：*0.3.0* 负责边栏分栏布局的纯函数：返回 `GutterColumns`，其中包含 `marks`、`braces`（每条轨道一个 `Band`，由内而外）与 `notes`。每个栏都是从轴线向外量得的 `Band(near, far)`，另有最远边缘 `extent`。

第 7 节　布局几何

标签会等其他内容全部绘制完毕后再放置，并采用以像素为单位、y 向上递增的显示坐标。`mosaickit.layout` 软件包收录布局所需的几何功能；它不导入任何渲染器，因此可单独测试及重复使用。本章的函数位于 `mosaickit.layout.geometry`；`Rect` 与 `polylabel` 也会从 `mosaickit.layout` 导出。点是数对 `(x, y)`，线段由一对点组成，多边形则是一串首尾相连的点。

7.1　矩形

Rect `Rect(x0, y0, x1, y1)`

闭合的轴对齐矩形 $[x_0, x_1] \times [y_0, y_1]$。`Rect.centered(center, width, height)` 以指定点为中心创建矩形。它提供 `width`、`height`、`center`、`inflate(pad)`、`corners()`（从 (x_0, y_0) 开始逆时针排列）、`edges()`、`contains(point)`（包含边界）、`within(other)`、`intersects(other)`（相切也算）与 `nearest_point(point)`。

引理 6 (矩形上的最近点) *对矩形 R 与点 p，`nearest_point` 返回的点*

$$q = \Big(\min(\max(p_x, x_0), x_1), \min\Big(\max\big(p_y, y_0\big), y_1\Big)\Big)$$

是 R 上离 p 最近的唯一点。

引线标注的引线会终止于这一点，因此标签位置一旦确定，引线便是最短的。

7.2　方向与线段

定义 7 (方向) 对平面上的点 a, b, c，

$$\operatorname{orient}(a,b,c) = (b_x - a_x)\big(c_y - a_y\big) - \big(b_y - a_y\big)(c_x - a_x),$$

即 $b-a$ 与 $c-a$ 的外积。

引理 8 (方向测试) 当 c 位于从 a 指向 b 的有向直线左侧（$a \to b \to c$ 为逆时针转向）时，$\operatorname{orient}(a,b,c)$ 为正；位于右侧时为负；a、b、c 共线时为零。它在循环置换下不变，交换两个参数时变号。

`segments_intersect` `segments_intersect(a, b, c, d) -> bool`
判断闭线段 $[a,b]$ 与 $[c,d]$ 是否有共同点。令 $d_1 = \operatorname{orient}(c,d,a)$、$d_2 = \operatorname{orient}(c,d,b)$、$d_3 = \operatorname{orient}(a,b,c)$、$d_4 = \operatorname{orient}(a,b,d)$；如果 $d_1 d_2 < 0$ 且 $d_3 d_4 < 0$，或某个 d_i 为零且其对应点落在另一线段的外接矩形（包住该线段的最小轴对齐矩形）内，便判定两者相交 (Cormen et al., 2009)。

定理 9 (线段相交) 在精确算术下，`segments_intersect(a, b, c, d)` 为真当且仅当 $[a,b] \cap [c,d] \neq \emptyset$。

`rect_hits_segment` `rect_hits_segment(rect, a, b) -> bool`
线段是否碰到矩形：有端点在矩形内，或线段与矩形四边之一相交。

引理 10 (线段与矩形) `rect_hits_segment(R, a, b)` 为真当且仅当 $[a,b] \cap R \neq \emptyset$。

7.3　多边形

定义 11 (简单多边形) 若多边形 $P = (p_0, \dots, p_{m-1})$（$m \geq 3$）的边 $[p_i, p_{i+1}]$（下标取模 m）只在相邻边的共同端点相交，且顶点不全共线，则称 P 为简单多边形。其边界 ∂P 是各边的并集；由 Jordan 曲线定理，∂P 的补集恰有一个有界连通分量，即内部 $\operatorname{int} P$，以及一个无界分量，即外部。记 $\overline{P} = \operatorname{int} P \cup \partial P$。

`polygon_edges` `polygon_edges(polygon) -> tuple[Segment, ...]`
`distance_to_boundary` `distance_to_boundary(point, polygon) -> float`
前者返回多边形的所有边（包括最后一点到第一点），后者计算指定点到最近边的欧氏距离。计算每条边的距离时，会先将点投影到该边所在的直线，再把参数限制在 $[0,1]$。

引理 12 (到线段的距离) 设 $a \neq b$，令

$$t^* = \operatorname{clamp}\big((p-a)\cdot(b-a)/\,|b-a|^2, 0, 1\big)$$

则 $a + t^*(b-a)$ 是 $[a,b]$ 上离 p 最近的点。

`point_in_polygon` `point_in_polygon(point, polygon) -> bool`
奇偶规则：从该点向右作水平射线，计算它穿过的边数。当一条边恰有一个端点严格位于射线所在直线上方，且交点在该点右侧时，该边计入 (Haines, 1994)。

命题 13 (奇偶规则) 设 P 为简单多边形且 $p \notin \partial P$。则 *`point_in_polygon(p, P)` 为真当且仅当* $p \in \operatorname{int} P$。

`rect_inside_polygon` `rect_inside_polygon(rect, polygon) -> bool`

`rect_overlaps_polygon` `rect_overlaps_polygon(rect, polygon) -> bool`
在内部：四个角都通过奇偶测试，且多边形没有任何边碰到矩形。重叠：有一个角通过测试，或有一条边碰到矩形（这也涵盖多边形整个位于矩形内的情形）。

定理 14 (多边形内的矩形) 设 P 为简单多边形、R 为矩形。则 *`rect_inside_polygon(R, P)` 为真当且仅当* $R \subset \operatorname{int} P$。

推论 15 (与多边形重叠的矩形) 设 P 为简单多边形、R 为矩形。则 *`rect_overlaps_polygon(R, P)` 为真当且仅当* $R \cap \overline{P} \neq \emptyset$。

`ray_exit` `ray_exit(origin, direction, polygon) -> float`
射线 $o + tr$ 与不平行于它的边相交处的最大 $t \geq 0$；没有这样的边时为 0。引线标注以它作为搜索起点，也就是射线离开区域之处。

命题 16 (永久离开多边形) 设 P 为简单多边形、$r \neq 0$，且 $T =$ *`ray_exit(o, r, P)`*。则对每个 $t > T$，$o + tr \notin \overline{P}$。

7.4　区域的视觉中心

区域的形心，也就是将区域视为均匀薄片时的质心，可能落在区域外（参见图 5）。标签应放在区域最深处，也就是离边界最远的点。

定义 17 (有符号距离与不可及极点) 对简单多边形 P，有符号距离定义为：当 $p \in \overline{P}$ 时 $f(p) = d(p, \partial P)$，否则 $f(p) = -d(p, \partial P)$。P 的不可及极点是 f 获取最大值 f^* 之处；f^* 即 P 内最大圆盘的半径。

引理 18 (有符号距离为 1-Lipschitz) 对所有点 p, q，$|f(p) - f(q)| \leq |p - q|$。

`polylabel` `polylabel(polygon, precision=1.0) -> Point`

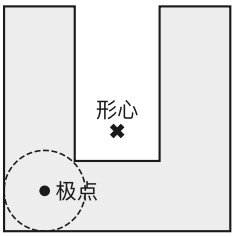


图 5
极点最大圆盘与形心

在正方形单元上的最佳优先搜索 (Agafonkin, 2016)。外接矩形先以边长等于其短边的正方形覆盖。中心为 c、半边长为 h 的单元得到上界 $f(c) + h\sqrt{2}$，放入优先队列，上界最大者先取出。程序记住目前最佳的中心，起始值为外接矩形的中心；取出的单元若上界比最佳值多出 `precision` 以上，就分成四格，否则舍弃。外接矩形宽或高为零的多边形返回其第一个顶点。

定理 19 (不可及极点) *设 P 为简单多边形，外接矩形的宽与高皆为正，并设精度 $\varepsilon > 0$。则 `polylabel` 必定结束，且返回的点 q 满足 $f(q) \geq f^{*} - \varepsilon$。特别地，当 $f^{*} > \varepsilon$ 时，q 位于* int P。

三角形的极点就是内心，也就是三条角平分线的交点。布局时会以默认的一像素精度调用 `polylabel`，这比任何文字的放置精度都更细。

7.5　大括号外形

`brace_outline`　`brace_outline(start, end, *, base, depth, direction, axis="y", samples=12)`

`Brace`

新增：*0.3.0*

覆盖 `start..end` 的大括号，以 `Brace(points, tip)` 表示。两端位于直线 `base`（固定的横向坐标），尖端位于跨距中点，在 `direction`（± 1）一侧距 `base` 为 `depth`。轴为 `"y"` 时各点为（横向, 纵向）；为 `"x"` 时为（纵向, 横向）。每段四分之一圆弧以 `samples` 段采样。空跨距、非正的深度，或其他方向与轴，引发 `ValueError`。

外形先在局部坐标中构造，u 为横向、v 为沿轴方向：四段半径 $r = \min\!\left(\frac{\text{depth}}{2}, \frac{\text{end} - \text{start}}{4}\right)$ 的四分之一圆，以两段直线相连，再沿横向拉伸 depth $/2r$ 倍。因此短跨距的圆弧较小，但尖端仍达到完整深度（参见图 6）。

命题 20 (大括号的形状) *设 $\ell < h$ 为跨距两端、$m = (\ell + h)/2$ 为中点、δ 为深度。在局部坐标中、采样之前，外形是从 $(0, \ell)$ 经尖端 (δ, m) 到 $(0, h)$ 的曲线，且 (i) 位于带状区 $0 \leq u \leq \delta$ 内；(ii) 在 $v \mapsto \ell + h - v$ 下对称；(iii) 除尖端外处处切线连续，尖端处曲线折返（尖点）。*



图 6
大括号外形

第 8 节　区域标签与点标签

标签用于命名已绘制的填色区域或点。标签图层会在所有其他图层之后，由延后处理程序绘制，因此能判断必须避开哪些对象。点标签先于区域标签放置；每个已放置的标签，都会成为后续标签的障碍物。

8.1　区域标签

`RegionLabelLayer`

```
RegionLabelLayer(region, text, short_text=None, placement="auto",
style=None,
                 stroke=Stroke(width=0.8), *, role="text", ...)
```

新增：*0.2.0*

为填色区域命名；区域可以是 `FillLayer` 的 id，也可以是至少由三点组成的多边形。`stroke` 设置引线标注的引线样式。

`placement="auto"`

如果文字放得下，就以极点为中心置于区域内（参见定义 17）；如果长文字放不下，再尝试 `short_text`。两者都放不下时，以 `text` 作引线标注。

`placement="inside"`

与 `"auto"` 相同，但两段文字都放不下时，仍把 `short_text`（或 `text`）置中于极点。

`placement="callout"`

一律作引线标注：区域外不加框的文字，以及从极点拉到文字的细引线。

“文字放得下”是指：文字矩形置中于极点、每边留白 2 pt 后，仍位于区域内（参见定理 14），且不碰到任何线条、标记、文字或其他区域。图 7 同时呈现这两种结果。

8.2　点标签

`PointLabelLayer`

```
PointLabelLayer(point, text, style=None, *, role="text", ...)
```

新增：*0.3.0*

在点的旁边加上文字为它命名，不使用引线。点的*足迹*是该处画有标记时的标记，否则就是点本身。

标签在足迹周围尝试 16 个方向，距足迹边缘 4、7、11、16 pt，由近而远。同一距离内根据定义 21 的顺序尝试方向（参见图 8），第一个不遮盖任何对象的位置胜出；若都不

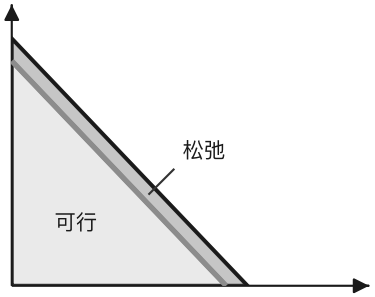


图 7
区域内标签与引线

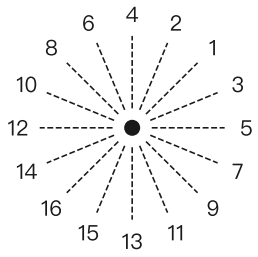


图 8
方向的尝试顺序

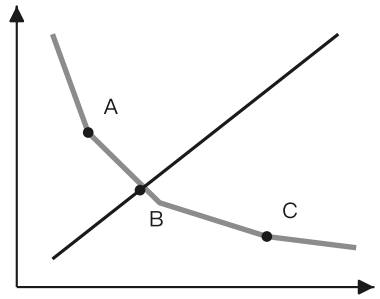


图 9
点旁的点标签

行，选违规数最少者（相同时取较早者），并发出指出该图层的 `LayoutWarning`。点严格位于其内部（距边缘超过 1.5 px）的区域是标签应在之处，不算障碍物。

定义 21 (候选顺序) 将方向自 $+x$ 起逆时针编号为 $k = 0, \dots, 15$，角度为 $2\pi k/16$。点标签按键值 $(\min(s, 16 - s), [s > 8])$（$s = (k - 2) \bmod 16$）递增的顺序尝试：按与右上方向 $k = 2$ 的角距离，角距离相同的两个方向则逆时针者优先。

8.3　硬性限制

两种标签都遵守同一条规则：不遮盖任何对象。障碍物包括坐标轴上已绘制的内容：路径的线段、标记与文字的矩形，以及填色区域的多边形。以下搜索函数位于 `mosaickit.layout.placement`；除 `place_beside` 外，也都由 `mosaickit.layout` 导出。它们与几何模块一样，以显示像素为单位运算；`scale` 则将以点为单位的间距换算为像素。

`Obstacles` `Obstacles(segments=(), rects=(), polygons=())`

`Placement` `Placement(rect, leader, violations)`

`Obstacles` 表示标签必须避开的障碍物；`Placement` 表示标签的放置结果，包括矩形、引线线段（不画引线的标签为 `None`）及违反的限制数。`Obstacles.extended(segments=..., rects=...)` 加入刚放置的标签。

定义 22 (违规数) 对候选矩形 R、障碍物 O、绘图区界线 B 与多边形集合 $\mathcal{P}$，*违规数*为

$$V(R) = [R \not\subseteq B] + \#\big\{s \in O_{\text{seg}} : R \cap s \neq \emptyset\big\} + \#\big\{Q \in O_{\text{rect}} : R \cap Q \neq \emptyset\big\} + \#\big\{P \in \mathcal{P} : R \cap \overline{P} \neq \emptyset\big\},$$

其中 $[\cdot]$ 在条件成立时为 1，否则为 0。$V(R) = 0$ 时称候选位置*不遮盖任何对象*。

各项分别由引理 10、`Rect.intersects` 与推论 15 精确计算。

8.4　引线标注

`place_callout`　`place_callout(polygon, size, obstacles, bounds, *, scale=1.0) -> Placement`

新增：*0.2.0*

在区域周围为给定大小的引线标注搜索位置。从极点 o 沿 16 个方向 r，把候选矩形放在距离 `ray_exit(o, r, P)` 再加 12、24 或 40 pt（近环）之处，并以面向极点的边或角对齐，使矩形朝远离区域的方向延伸。引线从极点拉向矩形上的最近点（参见引理 6），并在距该点 2 pt 处停止。候选位置的代价为 $V(R)$（区域本身也列入多边形），中心不在开阔空间时加一，引线离开自身区域后每穿过一条线、一段文字或一个其他区域加一，每位于一个交叉点之外（参见定义 23）再加一。

定义 23 (开阔空间与交叉点) *开阔空间*是在绘图区 4 pt 网格上、面积至少占十分之一的连通空白区域的并集；较小的空白区域称为*口袋*，例如两个阴影区域间未填色的窄带，文字放在那里看起来像在为口袋命名。区域的*交叉点*是两条障碍线段真正交叉（各自严格穿过对方）之处，且位于区域边界上或距边界 1.5 px 以内。从极点 o 看，若 $(p - x) \cdot (x - o) > 0$，则称点 p 位于 x *之外*。

越过交叉点后，线条朝远离区域的方向分开；若把标签放在那里，看起来会贴近线条的延伸方向，而不是所命名的区域。交叉点本身由方向测试求得。

引理 24 (交叉点) *若 $[a,b]$ 与 $[c,d]$ 真正交叉，则它们恰交于一点 $a + t(b-a)$，其中*

$$t = \frac{(c - a) \times (d - c)}{(b - a) \times (d - c)} \in (0, 1),$$

而 $u \times v = u_x v_y - u_y v_x$。

命题 25 (引线标注的选择) *以键值（代价, 引线长度）按字典序排列候选位置，并逐方向、由近到远的距离枚举。若近环中有代价为 0 的候选，`place_callout` 返回其中引线最短的第一个；否则再尝试远环（60 与 90 pt），返回两环中键值最小的第一个候选。结果只取决于参数。*

返回的代价不为零时，渲染器仍会在该处画出引线标注，并发出指出该图层的 `LayoutWarning`；放大画布或缩短文字通常就能消除。

fits_inside	`fits_inside(polygon, size, obstacles, bounds) -> Rect | None`
place_point_label	`place_point_label(footprint, size, obstacles, bounds, *, scale=1.0) ->`
place_beside	`Placement`
	`place_beside(anchor, size, *, axis, direction, reach, obstacles, bounds, scale=1.0)`

其他搜索：置中于极点的矩形，若放得下且除自身区域外不碰到任何对象就返回它；前述的点标签搜索；以及大括号尖端旁的搜索，在尖端外 3 到 45 pt 尝试，并沿跨距以 3 pt 为步长最多滑动 `reach`，位移小者优先。`place_beside` 返回矩形与其违规数。大括号标签在该处没有空位时，渲染器改从尖端尝试引线标注，保留违规较少者。

第 9 节　样式与颜色

9.1　稀疏样式

每个样式都是冻结的 dataclass，且所有字段都可以是 `None`。`None` 表示继承：字段值取自优先级较低的样式（详见第 10 节）。假值不等于 `None`，因此 `opacity=0`、`width=0` 与 `LegendStyle(visible=False)` 都是明确的覆盖。

定义 26 (稀疏合并) 设 a、b 为同类型、字段集合为 F 的样式。样式 $a \triangleright b$（`a.merged_over(b)`）对每个 $\varphi \in F$ 满足

$$(a \triangleright b)_\varphi = \begin{cases} a_\varphi \text{ if } a_\varphi \neq \text{None} \\ b_\varphi \text{ otherwise.} \end{cases}$$

空样式 ε 的每个字段都是 `None`。样式组逐槽合并，`None` 槽视同 ε。

命题 27 (合并构成幺半群) *对同类型的样式 a, b, c，* (*i*) *结合律为*

$$(a \triangleright b) \triangleright c = a \triangleright (b \triangleright c);$$

(*ii*) *空样式是单位元，*

$$\varepsilon \triangleright a = a \triangleright \varepsilon = a;$$

(*iii*) *合并具有幂等性，*

$$a \triangleright a = a.$$

逐栏来看，$a_1 \triangleright \cdots \triangleright a_n$ 取第一个不是 None 的值。

因此，多个样式无论如何分组合并，结果都相同。可以将它们视为一份优先级列表：某字段的值取自第一个设置该字段的来源。

SparseStyle　`style.merged_over(base)`
定义 26 的合并。不同类型的样式引发 `TypeError`。

9.2　样式类型

Stroke　`Stroke(color=None, width=None, dash=None, arrow=None, opacity=None)`
线条样式。`width` 以点为单位；`dash` 为 `DashStyle`（SOLID、DASHED、DOTTED、DASHDOT）；`arrow` 为 `ArrowStyle`（OPEN、TRIANGLE、FANCY、WEDGE），画在图层的 `ArrowPlacement`（START、END、BOTH）处。

DashStyle　`DashStyle.SOLID, DASHED, DOTTED, DASHDOT`
`Stroke` 的线条类型；也接受简写 `-`、`--`、`:` 与 `-.`。

SOLID　实线。

DashStyle
DASHED　虚线。

DashStyle
DOTTED　点线。

DashStyle
DASHDOT　点划线。

DashStyle

ArrowStyle　`ArrowStyle.OPEN, TRIANGLE, FANCY, WEDGE`
`Stroke` 的箭头样式。`SIMPLE` 是 `OPEN` 的另一个名称，也接受 Matplotlib 写法 `->` 与 `-|>`。

OPEN　开口箭头。

ArrowStyle
TRIANGLE　实心三角形。

ArrowStyle
FANCY　弯曲、渐细的箭头。

ArrowStyle
WEDGE　细楔形箭头。

ArrowStyle

ArrowPlacement　`ArrowPlacement.START, END, BOTH`
图层的箭头画在哪里。

START　在第一个点。

ArrowPlacement
END　在最后一个点。

ArrowPlacement
BOTH　两端都画。

ArrowPlacement

Fill　`Fill(color=None, opacity=None, hatch=None)`
区域内部的填色样式；`hatch` 为 Matplotlib 的填充图案，例如 `"//"`，`""` 表示无填充图案。

`Marker` `Marker(color=None, size=None, shape=None, opacity=None, edge_color=None, edge_width=None)`
点标记样式。`size` 是以平方点计的面积，与 Matplotlib 的 `scatter` 相同（36 对应 6 pt 圆点）；`shape` 为 Matplotlib 标记，例如 `"o"`、`"s"` 或 `"X"`。

`TextStyle` `TextStyle(color=None, size=None, family=None, weight=None, opacity=None, rotation=None)`
文字。`size` 以点为单位，`family` 为字体家族名称，`weight` 例如 `"bold"`，`rotation` 为逆时针角度（度）。

`LegendStyle` `LegendStyle(visible=None, location=None, frame=None, size=None)`
图例：Matplotlib 的位置名称，例如 `"best"` 或 `"upper right"`、外框，以及字号。

大小、宽度与不透明度在创建样式时检查：大小必须为有限非负数，不透明度在 $[0,1]$ 内，旋转角度必须有限。

表 4
基本默认值

样式	值
`Stroke`	`grey-800`、宽 1.5、实线、不透明度 1
`Fill`	`grey-200`、不透明度 0.3、无填充图案
`Marker`	`grey-800`、大小 36、`"o"`、不透明度 1、边框 `grey-800` 宽 0
`TextStyle`	`grey-900`、12 pt、DejaVu Sans、一般字重、不透明度 1、不旋转
`LegendStyle`	显示、位置 `"best"`、无外框、10 pt

9.3　颜色

`Color` `Color(red, green, blue, alpha=1.0)`

`TRANSPARENT` `Color.from_hex("#RGB" | "#RRGGBB" | "#RRGGBBAA")`
不可变的 RGBA 颜色，各通道在 $[0,1]$ 内。`channels` 返回四个值，`from_channels()` 创建颜色，`to_hex(include_alpha=None)` 写出 `#RRGGBB`，当 `include_alpha` 为真、或默认情况下 alpha 不为 1 时加上 `AA`。`TRANSPARENT` 为 `Color(0, 0, 0, 0)`。

命题 28 (十六进制往返) *对每个由十六进制数字组成、形如 `#RRGGBB` 或 `#RRGGBBAA` 的字符串 h，`Color.from_hex(h).to_hex(include_alpha=len(h) == 9)` 等于 h 的大写形式。三位数的 `#RGB` 读作 `#RRGGBB`。*

样式的 `color` 或 `edge_color` 接受 `Color`、`"#hex"` 字符串（立即解析），或其他任何字符串，后者保留为*调色板名称*。

9.4　调色板

`Palette` `Palette(name, colors)`

`DEFAULT_PALETTE`

新增：*0.2.0* 命名的颜色表。值可以是 `Color` 或十六进制字符串；`palette[name]` 查询颜色，名称不存在时引发指出该调色板的 `ConfigurationError`，`name in palette` 检查名称是否存在。

表 5 DEFAULT_PALETTE

名称	值	默认主题中的用途
grey-900	#222222	文字
grey-800	#333333	线条、标记、坐标轴
grey-600	#666666	坐标轴注释
grey-400	#999999	辅助线
grey-200	#CCCCCC	填色
grey-100	#E6E6E6	
white	#FFFFFF	画布背景
blue	#01A2D9	primary
red	#E3120B	secondary
teal	#00887D	accent

主题与样式以名称引用颜色，画布创建渲染计划时才在生效的调色板（`Config.palette`）中查询，因此渲染器只会收到具体颜色。在调色板中改一次颜色，所有引用它的角色都随之改变。调色板缺少的名称在绘制时引发 `ConfigurationError`，信息指明角色、样式字段与调色板。Python 的 `Palette` 会取代默认调色板，因此应从 `DEFAULT_PALETTE.colors` 创建，以保留内置主题所用的名称：

```
from mosaickit import DEFAULT_PALETTE, Canvas, Config, Palette,
use_config

brand = Palette("brand", {**DEFAULT_PALETTE.colors, "blue": "#0072B2"})
with use_config(Config(palette=brand)):
    canvas = Canvas()          # primary now draws in #0072B2
```

第 10 节　主题与配置

10.1　样式组与主题

StyleBundle `StyleBundle(stroke=None, fill=None, marker=None, text=None, legend=None)`
每个槽位包含一个稀疏样式。槽位内类型错误时，会引发 `ConfigurationError`。`merged_over(base)` 会逐槽位合并（参见定义 26）。

Theme `Theme(name, roles)`
名称，以及从角色名称到样式组的不可变对应。角色名称是非空、以点分隔的字符串（`"axes"`、`"axes.note"`、`"mypkg.boundary"`）。`with_roles(**patch)` 返回新主题，每个修补的角色合并在原样式组之上；关键字名称不能含点，因此带点的角色写成 `with_roles(**{"mypkg.line": bundle})`。`resolve(role, fallback_category=...)` 仅按主题解析角色。

表 6　default 主题

角色	样式组
primary	线条 blue
secondary	线条 red
accent	线条 teal
guide	线条 grey-400、虚线
axes	线条 grey-800、宽 1
axes.note	文字 9 pt、grey-600
canvas	填色 white、不透明度 1（背景）

其余的值都来自表 4 的基本默认值。此主题为 mosaickit.themes.default。

10.2　角色解析

解析图层角色时，会依次查看该角色及其上层角色，最后才查看图层类型的回退类别。该过程按字段进行；只有该角色与所有上层角色都未提供某字段的值时，才会使用回退类别的值。

定义 29 (角色链) 对回退类别为 φ 的角色 $\rho = \rho_1.\rho_2\cdots\rho_m$，其链为序列

$$\kappa(\rho) = (\rho_1\cdots\rho_m, \rho_1\cdots\rho_{m-1}, \ldots, \rho_1, \varphi),$$

由最具体到最一般，重复的键只保留第一次出现。

例如，文字图层的 "axes.note" 角色链为（axes.note, axes, text）。回退类别包括 primary（路径、群组）、region（填色）、point（标记）、text（文字、标签、坐标轴标记、注释、大括号）、annotation（箭头）与 legend。

定理 30 (解析顺序) *设某图层的自带样式为 E、角色链为 $\kappa = (k_1, \ldots, k_n)$，画在角色覆盖为 C 的画布上，所在配置的角色覆盖为 G、主题角色为 T，并令 D 为基本默认值。则图层解析后样式的每个字段，是序列*

$$E,\quad C[k_1], \ldots, C[k_n],\quad G[k_1], \ldots, G[k_n],\quad T[k_1], \ldots, T[k_n],\quad D$$

中第一个不是 None 的值，其中不存在的角色视为空样式组。

推论 31 (覆盖优先于具体程度) *画布或配置对一般角色的覆盖，优先于主题对较具体角色的设置：若 G[text] 设置了文字大小，且定理 30 序列中在它之前没有来源设置大小，则角色为 axes.note 的文字图层采用该大小，而非主题的 9 pt。*

本手册的图就依赖推论 31：它们同时覆盖 text、axes 与 axes.note 的文字大小，因为只覆盖 text 也会放大注释。

10.3　注册领域角色

ThemeRegistry　`ThemeRegistry()`

主题注册表，一开始就含有 `default`。`register(theme)` 加入主题（名称已被使用时引发 `ConfigurationError`），`get(name)` 查询主题，`register_roles(name, roles)` 把带点的领域角色合并到已注册的主题并返回它；不带点的角色引发 `ConfigurationError`，因此领域软件包无法遮蔽内置角色。

RolePack　`expand_roles(pack) -> dict[str, StyleBundle]`

expand_roles

以类型描述领域软件包角色的方式。`RolePack` 是带有类变量 `_namespace` 的 dataclass；每个不为 `None` 的字段成为角色 `namespace.field`，字段名称中的底线转为点。

```
from dataclasses import dataclass
from typing import ClassVar
from mosaickit import Stroke, StyleBundle, Theme, expand_roles
from mosaickit.themes import default

@dataclass(frozen=True)
class MyRoles:
    _namespace: ClassVar[str] = "mypkg"
    line: StyleBundle | None = StyleBundle(stroke=Stroke(color="blue",
width=2))
    line_dashed: StyleBundle | None = None

roles = expand_roles(MyRoles())        # {"mypkg.line": ...}
mytheme = Theme("mypkg", {**default.roles, **roles})
```

10.4　配置

Config　`Config(theme=default, canvas_spec=CanvasSpec(), renderer="matplotlib",`
`role_overrides={}, palette=DEFAULT_PALETTE)`

新画布的运行时默认值：主题、规格、渲染器（名称或 `Renderer`）、在主题之后应用的角色覆盖（定理 30 中的 G），以及调色板。

use_config　`with use_config(config): ...`

在块内使 `config` 成为生效的配置。此值存于上下文变量（context variable），因此每个线程与异步任务各自看到自己的配置，而传给 `Canvas` 的参数一律优先。

Config.load　`Config.load(path)`

Config.from_dict　`Config.from_dict(data, *, source="<dict>")`

读取严格的 TOML 配置。最上层的键为 `theme`（只能是 `"default"`；自定义主题须在 Python 中传入）、`canvas`（`CanvasSpec` 的字段）、`renderer`（只能是 `"matplotlib"`）、`palette` 与 `styles`。其他键、未知的样式、错误的值或无法读取的文件，都引发指明文件与键的 `ConfigurationError`。

```
[canvas]
x_range = [0, 20]
dpi = 150

[palette]
```

```
blue = "#0072B2"      # override a default color
accent = "#984EA3"    # add a new name

[styles."mypkg.boundary".stroke]
color = "accent"      # a palette name or a "#hex" value
width = 2
dash = "dashed"
```

`[palette]` 表叠加在 `DEFAULT_PALETTE` 之上，因此在此覆盖 `blue` 会改变 `primary` 以及所有引用 `blue` 的地方。样式的 `color` 与 `edge_color` 可以是调色板名称；未知名称在加载文件时（任何画布绘制之前）就引发指明文件与键的 `ConfigurationError`。

第 11 节　参数、网格与动画

11.1　参数与表达式

图层坐标不一定是数字，也可以是由命名参数组成的表达式。此时，场景描述一族图形，每组参数值对应一张图；绑定参数就是从中选出一张。

`Parameter`　`Parameter(name, value_type=None)`

命名的占位符。设置 `value_type` 后，绑定值必须是该类型的实例；所有绑定值都必须可哈希。`values(seq)` 返回 `ParameterValues`，并立即检查每个值。

`Expression`　`Constant(value)`

`Constant`

不可变的表达式树。参数与常数可用 `+`、`-`、`*`、`/`、`**`、一元 `-`，以及比较运算 `<`、`<=`、`>`、`>=` 与 `equals()` 组合；一般数字会包装成常数。表达式的相等是结构相等（`==` 比较两棵树，因此改用 `equals()` 创建比较表达式），把表达式当成布尔值使用会引发 `BindingError`：须先求值。`evaluate(bindings)` 根据参数到值的对应计算结果；`free_parameters()` 是树中所含参数的集合。

定义 32 (表达式与绑定) *表达式*是常数、参数，或对表达式 e_1, e_2 与运算 $\circ$ 而言的 $e_1 \circ e_2$。其自由参数满足

$$\operatorname{free}(c) = \emptyset, \quad \operatorname{free}(p) = \{p\},$$

以及

$$\operatorname{free}(e_1 \circ e_2) = \operatorname{free}(e_1) \cup \operatorname{free}(e_2).$$

绑定 β 是从参数到值的有限对应。若 $\operatorname{free}(e) \subseteq \operatorname{dom}\beta$，则 $\operatorname{bind}(e,\beta)$ 为值 $e(\beta)$；若 e 是未绑定参数，则为 e 本身；其余情况为

$$\operatorname{bind}(e_1,\beta) \circ \operatorname{bind}(e_2,\beta).$$

一般值 v 没有自由参数，求值结果为其本身。

定理 33 (部分绑定) *对每个表达式 e 与绑定 β，* (i)

$$\operatorname{free}(\operatorname{bind}(e,\beta)) = \operatorname{free}(e) \setminus \operatorname{dom}\beta;$$

(ii) *对每个满足*

$$\operatorname{dom}\gamma \cap \operatorname{dom}\beta = \emptyset$$

以及

$$\operatorname{dom}\gamma \supseteq \operatorname{free}(e) \setminus \operatorname{dom}\beta$$

的绑定 γ，都有

$$\operatorname{bind}(e,\beta)(\gamma) = e(\beta \cup \gamma).$$

推论 34 (分阶段绑定) *对定义域不相交的绑定 β_1, β_2，$\operatorname{bind}(\operatorname{bind}(e,\beta_1),\beta_2)$ 与 $\operatorname{bind}(e,\beta_1 \cup \beta_2)$ 有相同的自由参数，且在这些参数的每个绑定下有相同的值。特别地，`canvas.bind(p, 1).bind(q, 2)` 与 `canvas.bind({p: 1, q: 2})` 画出相同的图。*

`bind` 会遍历 tuple、mapping 与所有 `mosaickit` dataclass，因此整个场景一次绑定；图层的 `model` 保持不变。绘制仍含自由参数的场景时，引发指出这些参数的 `BindingError`。

```
from mosaickit import Canvas, Parameter, TextLayer

x = Parameter("x", value_type=float)
template = Canvas().add(TextLayer((x, 2 * x + 1), "moving"))
frame = template.bind(x, 3.0)
print(frame.snapshot().layers[0].position)    # (3.0, 7.0)
```

11.2　网格

CanvasGrid　`CanvasGrid(cells, rows=None, cols=None, shape=None, links=())`

Span　`Span(canvas, rows=1, cols=1)`

在一张图中放多个画布。`cells` 可以是画布、`Span` 与 `None`（空格）组成的平面列表，逐行填入；也可以是行的列表，此时允许跨行，且每行必须覆盖所有列。`shape=(rows, cols)` 等同同时传入两者。每个画布都会复制，之后修改它不影响网格。跨格重叠、跨格超出边界、单元过多，或混用平面与嵌套单元，都引发 `ConfigurationError`。`render()` 与 `save()` 的用法与画布相同。

命题 35 (推断的网格形状) *对 $n \geq 1$ 个一般单元组成的平面列表，若未给 `rows` 与 `cols`，网格有 $c = \lceil \sqrt{n} \rceil$ 列、$r = \lceil n/c \rceil$ 行。此时 $rc \geq n$、$r \leq c$，且空格少于 c 个，全部位于最后一行。*

只给 `cols` 时 $r = \lceil n/c \rceil$；只给 `rows` 时 $c = \lceil n/r \rceil$。

Layout　`CanvasGrid.from_layout(canvases, layout)`

CanvasGrid.from_layout

常见的排列：`SINGLE`、`STACKED`（两行）、`SIDE_BY_SIDE`、`GRID_2X2`、`GRID_3X3`，以及三个画布的 `TOP_TWO_BOTTOM_ONE` 与 `TOP_ONE_BOTTOM_TWO`，其中单独的画布横跨两列。

CanvasGrid.sweep　`CanvasGrid.sweep(template, values, *, cols=None)`

`ParameterValues` 的每个值一格，每格是绑定该值的模板。

GridLink　`GridLink(start_cell, start, end_cell, end, role="link", stroke=None)`

新增：*0.5.0*

从某格的 `start` 到另一格的 `end` 的直线，两点各以所在单元的数据坐标表示，画在整张图上、横越单元间的空隙。单元按布局顺序编号。样式是在起点单元的主题中解析的 `role`，再叠上 `stroke`。单元编号超出范围时，在创建网格时引发 `ConfigurationError`。

```
from mosaickit import Canvas, CanvasGrid, GridLink, MarkerLayer,
Parameter, Stroke

shift = Parameter("shift", value_type=float)
template = Canvas().add(MarkerLayer([(5, 2.5 + shift)]))
cells = [template.bind(shift, v) for v in (0.0, 2.0, 4.0)]
link = GridLink(0, (5, 2.5), 2, (5, 6.5), stroke=Stroke(dash="dashed"))
CanvasGrid(cells, cols=3, links=[link]).save("sweep.pdf")
```

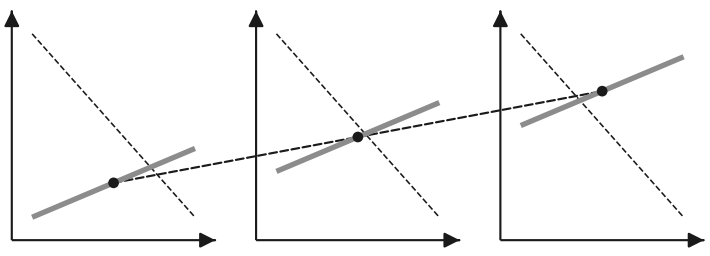


图 10
三个值的扫描

11.3　动画

Animation　`Animation(template, parameter, values, fps=30)`

`Animation.sweep(canvas, values, *, fps=30)`

每个值对应一帧，由模板绑定该参数值而成。`frames()` 以与渲染器无关的场景逐一产生帧；`save(path)` 写出 GIF（通过 Pillow）或 MP4（通过 `ffmpeg`）。创建动画时即按参数检查各值；值列表为空或 `fps` 非正时引发 `ConfigurationError`。

```
from mosaickit import Animation

values = shift.values([0.0, 1.0, 2.0, 3.0])
Animation.sweep(template, values, fps=4).save("sweep.gif")
```

第 12 节　绘制

12.1　渲染器

Renderer　`name: str`

`render(scene, context) -> Any`

`save(result, target, options) -> list[Path]`

所有后端都必须实现的协议。渲染器接收不可变的场景与私有的渲染上下文（规格、主题、覆盖、调色板、缓存与绑定），并返回由 `save` 写出的结果。网格与动画另需 `render_grid` 与 `save_animation`；渲染器如果没有实现它们，会引发 `RenderError`。内置后端仍在演进，因此渲染计划与上下文维持私有。

RendererRegistry　`RendererRegistry()`

`register(renderer)` 以 `name` 注册后端；如果对象缺少 `name`、`render` 或 `save`，或名称已被使用，会引发 `RenderError`。`get(name)` 返回已注册的渲染器，或加载内置渲染器。唯一的内置渲染器是 `"matplotlib"`，在第一次使用时才导入 (Hunter, 2007)。

12.2　Matplotlib 渲染器

绘制画布时先创建渲染计划：绑定场景（有自由参数时引发 `BindingError`）、展开群组、按 `z_index` 排序图层、解析每个图层的样式（参见定理 30），并把调色板名称换成颜色。图形采用画布的尺寸与 DPI，背景为 `canvas` 角色的填色，坐标轴设为规格的范围并关闭 Matplotlib 自身的轴线；mosaickit 中的坐标轴是图层。

接下来由各图层类型的构建函数依次绘制图层。需要参考其他所有内容的图层类型（图例、点标签、区域标签、边栏文字、跨距大括号）会在后续的延后处理阶段绘制；各阶段按注册顺序运行，并一次接收该类型的所有图层。

register_builder　`register_builder(layer_type, builder)`

register_pass　`register_pass(layer_type, run)`

新增：*0.2.0*

位于 `mosaickit.rendering.matplotlib`：不修改 mosaickit 就让渲染器识别新的图层类型。构建函数以 `builder(ax, resolved)` 调用并返回 Matplotlib artist；处理阶段以

`run(ax, layers, context)` 调用，`PassContext` 的 `handles` 把图层 id 对应到图例用的 artist。`resolved.layer` 是图层，`resolved.style` 是解析后的 `StyleBundle`。查询按方法解析顺序进行，子类因此继承父类的注册。图层以类变量 `style_slots` 声明每个槽的自带样式存于哪个字段，以 `fallback_category` 声明最后依据的角色。

区域标签的引线标注会避开坐标轴上的所有内容，包括第三方注册的构建函数所画的东西。

12.3　结果与保存

`SaveOptions`　`SaveOptions(transparent=False, expand=True)`

结果的写出方式；`canvas.save(path, **options)` 把关键字参数传到这里。`transparent` 去除背景。`expand=True` 时，保存会把画布扩大到恰好容纳画到边缘外的内容（例如边栏文字），另加 4 pt 留白；它从不裁切，因此放得下的图维持 `CanvasSpec` 指定的尺寸。`expand=False` 保持指定尺寸。

```
  result = canvas.render()
  try:
      result.show()          # an interactive window
  finally:
      result.close()
```

`render()` 返回 `MatplotlibResult`，提供 `figure`、`axes`、`show()`、`save(target, **options)` 与 `close()`。`canvas.save()` 会自行关闭暂时的结果。文件格式按扩展名决定：`.png`、`.pdf` 或 `.svg`（其他扩展名引发 `RenderError`）；动画接受 `.gif` 或 `.mp4`。网格中每格大小相同：宽为各画布宽度除以所跨列数的最大值，高为各画布高度除以所跨行数的最大值；DPI 取各画布中的最高者。

12.4　缓存

`RenderCache`　`RenderCache(max_entries=256)`

`CacheKey`

有容量上限、线程安全的最近最少使用缓存，存放解析后的图层，以图层 id、绑定、模型、规格与样式为键。除非以 `cache=` 传入，每次绘制、网格或动画都创建自己的缓存，因此任务之间不会意外共用状态；要在多个任务间重用结果时，传入同一个缓存。无法哈希的模型不经缓存绘制，每种模型类型在每个缓存中只发出一次 `CacheBypassWarning`；模型请使用冻结的 dataclass。`clear()` 清空缓存。

附录 A　证明

本附录按各章出现的顺序，证明其中的引理、命题、定理与推论。以下假设精确算术。代码以浮点数计算相同的量，因此若某个几何构型与退化情形的差异仅在舍入误差范围内（例如点落在边上，或三点几乎共线），代码可能将它判定为任一种结果。平面向量 u, v 的外积记为 $u \times v = u_x v_y - u_y v_x$。

A.1　排列与轨道

定理 2 的证明： 变量代换。令 $o_k = \sum_{j<k}\big(s_j + g\big) + s_k/2$、$y_k = x_k - o_k$、$z_k = c_k - o_k$。由于 $o_{k+1} - o_k = (s_k + s_{k+1})/2 + g$，定义 1 的限制条件即 $y_1 \le \cdots \le y_n$，目标函数为 $\sum_k (y_k - z_k)^2$。因此问题是在最小二乘意义下，找出最接近 z 的非递减向量。

*代码的计算。*考察从第 i 项开始、起点为 S_B 的连续项目组 B。对于 $k \in B$，代码将它放在

$$x_k = S_B + \sum_{i \le l < k} (s_l + g) + s_k/2 = S_B + o_k - O_i$$

其中 $O_i = \sum_{l<i}(s_l + g)$。因此 $y_k = S_B - O_i =: Y_B$ 在 B 上为常数。代码取 S_B 为 $c_k - (o_k - O_i)$ 的平均，使 Y_B 成为 z_k 在 B 上的平均；新加入的单一项目则有 $Y = z_k$。组 A 与后续组 B 在 $S_A + \mathrm{length}(A) + g \le S_B$ 时保持分开。由于 $\mathrm{length}(A) + g = O_{i_B} - O_{i_A}$，此条件等价于 $Y_A \le Y_B$。因此，代码实现的就是 PAVA：将 z_k 加入为一个块；只要最后两个块的平均递减，就以它们的并集（取其平均）取代。

不变式。(a) 相邻块的平均非递减：最后一对顺序正确时合并即停止，较早的块对不受影响。(b) 在平均为 μ_B 的每个块 B 中，每个起始片段的平均至少为 μ_B。单一项目满足 (b)。A 与 B 合并时 $\mu_A > \mu_B$，合并后的平均 μ 严格介于两者之间。A 内的起始片段平均至少为 $\mu_A > \mu$。其他起始片段是 A 接上 B 的起始片段 B'；若 B' 为整个 B，其平均为 μ；否则 B 的其余部分 B'' 由 B 的 (b) 知平均最多 $\mu_B < \mu$，而该片段是平均为 μ 的集合中 B'' 的补集，平均至少为 μ。

*最佳性。*目标函数严格凸、限制条件 $y_k - y_{k+1} \le 0$ 为线性，因此满足 Karush–Kuhn–Tucker 条件的点即唯一的最小解 (Boyd & Vandenberghe, 2004)。对 $k = 0, \ldots, n$ 令 $\lambda_k = 2\sum_{l \le k}(z_l - y_l)$。平稳条件 $2(y_k - z_k) + \lambda_k - \lambda_{k-1} = 0$ 由构造成立。块内的偏差 $z_l - Y_B$ 总和为零，因此 λ 在每个块末端为零，包括 $\lambda_n = 0$。在由第 i 项起的块内，由 (b) 得 $\lambda_k = 2\sum_{l=i}^{k}(z_l - Y_B) \ge 0$。最后，$\lambda_k > 0$ 只发生在 k 与 $k+1$ 属于同一块时，此时 $y_k = y_{k+1}$，限制条件取等号。加上 (a) 给出的可行性，所有条件都成立。 □

推论 3 的证明： (i) 按排序后的顺序编号，并设 $i < j$。把第 i 到第 $j-1$ 条限制相加，得

$$x_j - x_i \ge \sum_{k=i}^{j-1}\big((s_k + s_{k+1})/2 + g\big) \ge \big(s_i + s_j\big)/2 + g$$

因为每项都非负，且首末两项分别贡献 $s_i/2$ 与 $s_j/2$。(ii) 若 c 是排列，它使目标函数为 0，因此是定理 2 的唯一最小解。(iii) 沿用该证明的记号，

$$\sum_{k \in B}(x_k - c_k) = \sum_{k \in B}(y_k - z_k) = |B|\, Y_B - \sum_{k \in B} z_k = 0$$

□

定理 5 的证明： 区间只在与某轨道上所有区间都不冲突时才加入该轨道，之后不再移动，因此结果是轨道分配。任何轨道分配都必须把 ω 个两两冲突的区间放在 ω 条不同轨道，所以至少需要 ω 条轨道。

反过来，记 $J=[a_J,b_J]$，并设区间按 a 递增的顺序给出。由定义 4，I 与 J 冲突恰好等价于半开区间 $[a_I,b_I+g)$ 与 $[a_J,b_J+g)$ 相交。设 I 被放在所用的最高轨道 k。I 到来时，每条轨道 $j<k$ 上都有与 I 冲突的区间 J_j。每个 J_j 较早到来，故 $a_{J_j}\le a_I$；又与 I 冲突，故 $b_{J_j}+g>a_I$。因此每个 $\left[a_{J_j},b_{J_j}+g\right)$ 都含 a_I，而 $b_I-a_I+g>0$ 使 $[a_I,b_I+g)$ 也含 a_I。有共同点的半开区间两两相交，所以 $J_0,\dots,J_{k-1},I$ 是 $k+1$ 个两两冲突的区间，$k+1\le\omega$。 □

A.2　矩形、线段与多边形

引理 6 的证明： $|q-p|^2=(q_x-p_x)^2+\left(q_y-p_y\right)^2$，而 R 是 $[x_0,x_1]$ 与 $[y_0,y_1]$ 的乘积，因此两个坐标可分别最小化。在区间上，$t\mapsto(t-p_x)^2$ 严格凸，无限制的最小值在 p_x，所以它在 $[x_0,x_1]$ 上的最小值只在 $\min(\max(p_x,x_0),x_1)$ 获取；y 亦同。 □

引理 8 的证明： 将其他行减去第一行，得

$$\operatorname{orient}(a,b,c)=\det\begin{pmatrix}1 & a_x & a_y\\ 1 & b_x & b_y\\ 1 & c_x & c_y\end{pmatrix}.$$

列的循环置换为偶置换，交换两行为奇置换，由此得到对称性。又

$$\operatorname{orient}(a,b,c)=(b-a)\times(c-a)=|b-a|\,|c-a|\sin\theta$$

其中 θ 为从 $b-a$ 到 $c-a$ 的有号角；它为正恰好在 $\theta\in(0,\pi)$，即 c 位于有向直线左侧时，位于右侧时为负，而在两向量平行或其一为零，即三点共线时为零。 □

定理 9 的证明： 先说明：若 $\operatorname{orient}(c,d,p)=0$ 且 p 位于 $[c,d]$ 的外接矩形内，则 $p\in[c,d]$。事实上，若 $c=d$，外接矩形就是点 c；否则由共线性，对某实数 t 有 $p=c+t(d-c)$，而在 $d-c$ 不为零的坐标上，外接矩形条件迫使 $t\in[0,1]$。

*充分性。*设 $d_1d_2<0$ 且 $d_3d_4<0$。$\operatorname{orient}(c,d,\cdot)$ 沿 $[a,b]$ 为仿射函数且变号，因此 $[a,b]$ 与直线 L_{cd} 交于一点；同理 $[c,d]$ 与 L_{ab} 交于一点。由于 a、b 位于 L_{cd} 两侧，两直线不平行，恰有一个共同点 z；两个交点都等于 z，因此 z 同时位于两线段上。另一种情形是某个 $d_i=0$ 且其端点位于另一线段的外接矩形内，由上述说明，该端点位于另一线段上。

*必要性。*设 z 为共同点。若 $d_1d_2>0$，则 a、b 严格位于 L_{cd} 的同一侧，整个 $[a,b]$ 亦然，与 $z\in L_{cd}$ 矛盾；所以 $d_1d_2\le0$，同理 $d_3d_4\le0$。若两者皆为负则得证。否则某个 d_i 为零；不妨设 $d_1=0$，其余情形对称。若 $d_2\neq0$，$[a,b]$ 只在 a 与 L_{cd} 相交，故 $z=a$ 且 $a\in[c,d]$：d_1 的测试成立。若 $d_2=0$，四点共线，两线段是同一直线上重叠的区间，其中一条的端点位于另一条内，因此四个测试之一成立。 □

引理 10 的证明： R 的边都在 R 内，因此任一测试成立都给出共同点（参见定理 9）。反之，设 $[a,b]$ 与 R 相交但两端点都不在 R 内。线段是连通的，同时含有 R 内与 R 外的点，因此与 R 的边界相交，而边界是四条边的并集；定理 9 会检测到该边。 □

引理 12 的证明：$t \mapsto |a + t(b-a) - p|^2$ 是凸二次函数，无限制的最小值在 $(p-a)\cdot(b-a)/\,|b-a|^2$；如引理 6 的证明，它在 $[0,1]$ 上的最小值位于该值的截断处。 □

命题 13 的证明：记 $p = (x, y)$，取 $\varepsilon > 0$ 小于所有顶点高度与 y 的正差值 $|y_i - y|$，且小到使 $p_\varepsilon = (x, y+\varepsilon)$ 与 p 位于 $\mathbb{R}^2 \setminus \partial P$ 的同一连通分量（因 $p \notin \partial P$ 且各分量为开集，此为可行）。直线 $Y = y + \varepsilon$ 上没有顶点，且对每个顶点，$y_i > y$ 当且仅当 $y_i > y + \varepsilon$。因此一条边通过代码的高度测试，恰好等价于它穿过直线 $Y = y + \varepsilon$，交点横坐标为

$$X_\varepsilon = x_0 + (y + \varepsilon - y_0)(x_1 - x_0)/(y_1 - y_0)$$

代码比较的是 x 与 X_0。若 $x = X_0$，由于高度测试使 y 介于两端点高度之间，点 (X_0, y) 位于该边上，即 $p \in \partial P$，与假设矛盾；所以 $x \neq X_0$，由连续性，对够小的 ε，$x < X_0$ 当且仅当 $x < X_\varepsilon$。因此代码计算的正是从 p_ε 向右、不经过任何顶点的射线所穿过的边。每次穿越都使射线在内部与外部之间切换，而射线在最右方位于外部，所以计数为奇数恰好在 $p_\varepsilon \in \operatorname{int} P$，即 $p \in \operatorname{int} P$ 时 (Haines, 1994)。 □

定理 14 的证明：若 $R \subset \operatorname{int} P$，则 R 与 ∂P 不相交，因此没有边碰到它（参见引理 10），而它的角位于 $\operatorname{int} P$ 且不在边界上，因此通过奇偶测试（参见命题 13）。反之，若没有边碰到 R，则 $R \cap \partial P = \emptyset$（参见引理 10）；特别是各角不在边界上，通过测试即表示它们位于 $\operatorname{int} P$。R 是连通的且与 ∂P 不相交，因此位于 $\mathbb{R}^2 \setminus \partial P$ 的一个分量内；它含有一个位于 $\operatorname{int} P$ 的角，所以 $R \subset \operatorname{int} P$。 □

推论 15 的证明：若有一个角通过测试，它不在边界上时位于 $\operatorname{int} P$（参见命题 13），否则位于 ∂P；无论哪种情形都有 $R \cap \overline{P} \neq \emptyset$。若有一条边碰到 R，则 R 与 ∂P 相交。反之，设 R 与 $\overline{P}$ 相交。若它与 ∂P 相交，就有某条边碰到它。否则 R 与 ∂P 不相交而与 $\operatorname{int} P$ 相交；由连通性得 $R \subset \operatorname{int} P$，其各角通过测试。 □

命题 16 的证明：设对某个 $t > T$，$p = o + tr \in \partial P$，位于边 e 上。若 e 不平行于 r，代码对 e 解 $o + tr = a + u(b-a)$，得到这个 $t \geq 0$ 与 $u \in [0,1]$，因此 $T \geq t$，矛盾。若 e 平行于 r，它位于射线所在直线 ℓ 上。取包含 e、位于 ℓ 上的极大连续边段。它不是整个 P（P 的顶点不全共线），所以边段两端都是与某条不在 ℓ 上、因而不平行于 r 的边共用的顶点。边段中相邻的边不会折返，因为从同一顶点沿 ℓ 同向出发的两条边会重叠，与 P 为简单多边形矛盾；因此边段是 ℓ 上的一段，其在 r 方向的远端是端点顶点 $v = o + t_v r$，且 $t_v \geq t > T$。相邻的不平行边含有 v（$u \in \{0,1\}$），因此 $T \geq t_v$，同样矛盾。所以 $\{o + tr : t > T\}$ 与 ∂P 不相交；它连通且无界，因此位于外部。 □

A.3　不可及极点

引理 18 的证明：对任意集合 S，由三角不等式得 $|d(p,S) - d(q,S)| \leq |p - q|$，这就处理了 p、q 在同一侧的情形。设 $p \in \overline{P}$、$q \notin \overline{P}$，并令 z 为从 p 到 q 的线段上，属于 $\overline{P}$ 的最

后一点（$\overline{P}$ 为闭集，故存在）。紧接在 z 之后的点都在外面，因此 $z \notin \operatorname{int} P$，即 $z \in \partial P$。于是

$$|f(p) - f(q)| = d(p, \partial P) + d(q, \partial P) \le |p - z| + |z - q| = |p - q|$$

□

定理 19 的证明：代码精确计算 f：不在边界上时，由命题 13 符号正确；在边界上时距离为 0。记 b 为目前的最佳值；它只会增加，且始终是某个已求值中心的 f 值。

结束性。每次分割使半边长 h 减半。取出 $h\sqrt{2} \le \varepsilon$ 的单元时，更新后 $b \ge f(c)$，因此其上界最多比 b 多 $h\sqrt{2} \le \varepsilon$，单元被舍弃而不分割。所以单元构成有限树，每格各推入与取出一次。

不变式。外接矩形的每一点都位于某格的闭正方形内，该格要不在队列中，要不在被舍弃当时满足 $f(c) + h\sqrt{2} \le b + \varepsilon$。初始正方形从左下角起以短边为步长铺设，直到越过远端边缘，覆盖了外接矩形；舍弃按舍弃规则保持不变式；分割把一个正方形换成覆盖它的四个正方形。

结论。结束时队列为空。设 p^* 为极点；它位于 $\overline{P}$ 中、外接矩形内，落在某个被舍弃、中心为 c、半边长为 h 的单元内。由引理 18 与 $|p^* - c| \le h\sqrt{2}$，

$$f^* = f(p^*) \le f(c) + h\sqrt{2} \le b + \varepsilon \le f(q) + \varepsilon,$$

其中 q 为返回的点，$f(q)$ 为最终的 b。若 $f^* > \varepsilon$，则 $f(q) > 0$，故 q 位于 $\overline{P}$ 内且与 ∂P 距离为正，即 $q \in \operatorname{int} P$。□

命题 20 的证明：记半径为 r，拉伸倍率为 $\sigma = \delta/2r \ge 1$。拉伸之前，曲线由以下部分组成：圆心 $(0, \ell + r)$、从 -90° 到 0° 的圆弧 A_1；从 $v = \ell + r$ 到 $v = m - r$ 的线段 $u = r$；圆心 $(2r, m - r)$、从 180° 到 90° 的圆弧 A_2；圆心 $(2r, m + r)$、从 270° 到 180° 的圆弧 A_3；从 $m + r$ 到 $h - r$ 的线段 $u = r$；以及圆心 $(0, h - r)$、从 0° 到 90° 的圆弧 A_4。（当 $r = (h - \ell)/4$ 时两线段长度为零。）曲线起于 $(0, \ell)$、终于 $(0, h)$，A_2 与 A_3 交于 $(2r, m)$，拉伸 $(u, v) \mapsto (\sigma u, v)$ 把它送到 (δ, m)。

(i) 在 A_1 与 A_4 上 $u \in [0, r]$；在两线段上 $u = r$；在 A_2 与 A_3 上 $u \in [r, 2r]$。拉伸后 $u \in [0, \delta]$。

(ii) 反射 $v \mapsto \ell + h - v$ 交换 A_1 与 A_4、A_2 与 A_3 的圆心，并把角度 θ 变为 $-\theta$，因此把 A_1 映到 A_4、A_2 映到 A_3，两线段互换。

(iii) 沿角度递增方向走圆弧时，运动方向为 $(-\sin\theta, \cos\theta)$；沿递减方向时为 $(\sin\theta, -\cos\theta)$。在 A_1 终点（$\theta = 0$）与 A_2 起点（$\theta = 180^\circ$）两者都是 $(0, 1)$，即两者间线段的方向；A_3 终点与 A_4 起点亦同。在尖端，A_2 以方向 $(1, 0)$ 抵达（$\theta = 90^\circ$），A_3 以 $(-1, 0)$ 离开（$\theta = 270^\circ$）：形成尖点。拉伸与最后的放置都是可逆仿射映射，会带着切线方向且保持其非零，因此连续性与尖点都得以保留。□

A.4　标签

引理 24 的证明： 共同点满足 $a + tr = c + us$，其中 $r = b - a$、$s = d - c$。两边与 s 作外积可消去 u：$t(r \times s) = (c - a) \times s$。真正交叉使 a、b 严格位于 L_{cd} 两侧，因此两直线不平行，$r \times s \neq 0$，交点唯一。由于 $\mathrm{orient}(c, d, \cdot)$ 沿 $[a, b]$ 为仿射函数且两端异号，其零点位于某个 $t \in (0, 1)$。 □

命题 25 的证明： 代码保留一个最佳候选，只在遇到键值严格较小的候选时才替换，因此在枚举的任何前段之后，它保存的是目前为止键值最小的第一个候选。近环结束后，若最小代价为 0，最小键值为 $(0, \ell_{\min})$，搜索停止：结果是引线最短的第一个零代价候选。否则以同一个最佳候选继续枚举远环，结果是两环中键值最小的第一个候选。每个步骤都是参数（极点、离开距离、开阔区域、交叉点）的函数，没有随机性，枚举顺序固定。 □

A.5　样式、主题与绑定

命题 27 的证明： 逐栏来看，合并是运算：$x \neq \mathrm{None}$ 时 $x \circ y = x$，否则为 y。$(x \circ y) \circ z$ 与 $x \circ (y \circ z)$ 都等于 x, y, z 中第一个不是 `None` 的值（或 `None`），`None` 是双边单位元，且 $x \circ x = x$。样式在各栏相等时相等，由此得 (i)–(iii)，再由归纳法得到第一个非 `None` 值的描述。样式组逐槽同理，`None` 槽的行为如同空样式。 □

命题 28 的证明： 一对十六进制数字 $k \in \{0, \dots, 255\}$ 读为 $k/255$，写出时取 $\mathrm{round}(255 \cdot k/255)$ 的两位大写数字。在双精度下，计算出的乘积与 k 相差最多 $255 \cdot 2^{-52} < 1/2$，因此舍入为 k。`include_alpha` 为真，即九个字符的形式时，才写出 alpha 那一对。三位数形式在读取前先把每位数字重复一次。 □

定理 30 的证明： 以 $\triangleright$ 表示合并。解析从 D 开始，按 T、G、C 的顺序，对每个对应依次把 $M[k_n], \dots, M[k_1]$（由最一般到最具体）合并到结果之上，不存在的键给出空样式组；最后合并图层自带的样式组 E，它由 `style_slots` 指出的字段构成。展开并以结合律（参见命题 27）去掉括号，解析后的样式组为

$$E \triangleright C[k_1] \triangleright \cdots \triangleright C[k_n] \triangleright G[k_1] \triangleright \cdots \triangleright T[k_n] \triangleright D,$$

其各字段为据此顺序第一个不是 `None` 的值。之后绑定调色板只把颜色名称换成颜色，不影响其他字段。 □

推论 31 的证明： 角色为 `axes.note` 的文字图层，其链为（`axes.note`, `axes`, `text`），在定理 30 的序列中 $G[\mathrm{text}]$ 位于 $T[\mathrm{axes.note}]$ 之前。由假设，更早的项目都没有设置大小，因此第一个非 `None` 的大小来自 $G[\mathrm{text}]$。 □

定理 33 的证明： 对 e 作归纳。若 $\mathrm{free}(e) \subseteq \mathrm{dom}\,\beta$（特别是常数，以及属于 $\mathrm{dom}\,\beta$ 的参数），$\mathrm{bind}(e, \beta)$ 是一般值 $e(\beta)$：它没有自由参数，符合 (i)，且求值为 $e(\beta) = e(\beta \cup \gamma)$，因为表达式的值只取决于其自由参数的值。若 e 是参数 $p \notin \mathrm{dom}\,\beta$，结果

为 p，$\mathrm{free} = \{p\} = \mathrm{free}(e) \setminus \mathrm{dom}\,\beta$，值为 $\gamma(p) = (\beta \cup \gamma)(p)$。否则 $e = e_1 \circ e_2$，结果为 $\mathrm{bind}(e_1, \beta) \circ \mathrm{bind}(e_2, \beta)$（一般值包装为常数）。其自由参数为

$$(\mathrm{free}(e_1) \setminus \mathrm{dom}\,\beta) \cup (\mathrm{free}(e_2) \setminus \mathrm{dom}\,\beta) = \mathrm{free}(e) \setminus \mathrm{dom}\,\beta$$

而 γ 绑定了两部分各自的自由参数，由归纳假设其值为

$$e_1(\beta \cup \gamma) \circ e_2(\beta \cup \gamma) = e(\beta \cup \gamma)$$

□

推论 34 的证明：两次应用定理 33 (i)，两者的自由参数都是 $\mathrm{free}(e) \setminus \mathrm{dom}\,(\beta_1 \cup \beta_2)$。对这些参数的绑定 γ（与 β_1、β_2 不相交），两次应用 (ii) 得

$$\mathrm{bind}(\mathrm{bind}(e, \beta_1), \beta_2)(\gamma) = \mathrm{bind}(e, \beta_1)(\beta_2 \cup \gamma) = e(\beta_1 \cup \beta_2 \cup \gamma) = \mathrm{bind}(e, \beta_1 \cup \beta_2)(\gamma)$$

画布以这种方式绑定其场景中的每个表达式。□

命题 35 的证明：由 $r = \lceil n/c \rceil$ 得 $rc \geq n > (r-1)c$，因此前 $r-1$ 行是满的，最后一行有 $n - (r-1)c \in [1, c]$ 格，空格数 $rc - n < c$。由 $c = \lceil \sqrt{n} \rceil$ 得 $n \leq c^2$，所以 $n/c \leq c$，$r \leq c$。□

更新记录

本章列出影响软件包使用的版本变更，不列仅修改文档的版本。按照 l3doc 惯例，每个条目都标注在它所描述的功能旁，因此所列页码就是该功能的实际页码。

v0.8.0

DashStyle：DashStyle 与 ArrowStyle 接受简写，让其他包可直接当作自己的线型与箭头枚举 ····· 19

v0.5.1 (2026-10-03)

MarkerLayer：绘图区边缘的标记完整画出；中心在绘图区外的标记略过 ····· 7

v0.5.0 (2026-10-03)

ArrowLayer：虚线、点线或点划线应用在箭身；箭头维持实线 ····· 7

CanvasGrid：新增 links ····· 26

v0.4.0 (2026-10-03)

BraceLayer：内侧大括号的标签在尖端外没有空位时改以引线拉出（先前会重叠并发出警告） ····· 8

v0.3.2 (2026-10-03)

PointLabelLayer：只有点严格位于区域内部时，该区域才算是点自己的区域；位于区域边上的点，标签仍放在区域外 ····· 15

v0.3.1 (2026-10-03)

PointLabelLayer：点标签可以位于包含其点的填色区域内 ····· 15

v0.3.0 (2026-10-03)

AxisSpec：坐标轴标题改放在箭头尖端之外，不再置中于尖端 ····· 8

mosaickit：坐标轴外的文字按栏排列：标记、外侧大括号（每条轨道一栏）、注释；标记与注释会沿轴分散，互不重叠 ····· 8

点标签先于区域标签放置，区域的引线标注因此会避开它们；标签文字按旋转角度测量 ···· 15

AxisNoteLayer：默认主题新增 axes.note 角色 ····· 8

Obstacles：Placement.leader 可以是 None ···· 16

fits_inside：新增 place_point_label 与 place_beside ····· 18

Palette：主题与样式以调色板名称引用颜色，在创建渲染计划时按 Config.palette 解析；未知名称引发指明角色、字段与调色板的 ConfigurationError ····· 21

Config：新增 Config.palette，默认为 DEFAULT_PALETTE ····· 23

Config.load：TOML 接受 [palette] 表，样式颜色可以是调色板名称，并在加载文件时检查 ····· 24

SaveOptions：新增 expand（默认 True） ··· 28

v0.2.0 (2026-10-03)

mosaickit：内置渲染器在第一次使用时按名称加载；核心不再导入 Matplotlib 后端 ····· 3

LayoutWarning：新增，在没有任何引线标注位置能避开所有障碍物时发出 ····· 6

PathLayer：新增 clip（默认 True）；箭头不再于路径末端重画实线，虚线箭头因此保持虚线 · 7

TextLayer：anchor 接受四个角 top-left、top-right、bottom-left、bottom-right ····· 7

ArrowLayer：箭头不再被坐标轴裁切 ····· 7

AxisSpec：坐标轴默认改画实心三角箭头；轴线不受裁切，在绘图区边缘保持完整宽度 ····· 8

mosaickit.layout：新增：标签布局背后的纯几何模块 ····· 11

DEFAULT_PALETTE：新增：灰阶、white、blue、red 与 teal；默认主题的颜色取自此调色板，primary、secondary 与 accent 改为蓝、红、青绿 ····· 21

resolve：themes.resolve 接受 overrides，在主题之后依次应用 ····· 23

Layer：图层声明 style_slots；Matplotlib 绘制改由各类型的注册表驱动 ····· 28

v0.1.1 (2026-10-01)

一般：PyPI 上的软件包信息链接首页、仓库、问题追踪、更新记录与发布说明 ····· 2

v0.1.0 (2026-10-01)

一般：首次发布：不可变的场景与图层、稀疏样式、具命名空间的主题、画布、可跨格的网格、参数表达式、动画、以任务为范围缓存的 Matplotlib 绘制，以及严格的 TOML 配置 ···· 2

命令索引

A

Animation ····· 27
ArrowLayer ····· 7
ArrowPlacement ····· 19
ArrowStyle ····· 19
assign_lanes ····· 10
AxisMarkLayer ····· 8
AxisNoteLayer ····· 8
AxisSpec ····· 8

B

box_frame ····· 8
Brace ····· 14
brace_outline ····· 14
BraceLayer ····· 8
build_axes ····· 8

C

CacheKey ····· 28
Canvas ····· 5
CanvasGrid ····· 26
CanvasGrid.from_layout ····· 26
CanvasGrid.sweep ····· 26
CanvasSpec ····· 5
Color ····· 20
Config ····· 23
Config.from_dict ····· 23
Config.load ····· 23
Constant ····· 24
crosshair_axes ····· 8

D

DashStyle ····· 19
DEFAULT_PALETTE ····· 20
distance_to_boundary ····· 12

E

expand_roles ····· 23
Expression ····· 24

F

Fill ····· 19
FillLayer ····· 7
fits_inside ····· 18

G

GridLink ····· 26
GroupLayer ····· 8
gutter_columns ····· 11

I

Interval ····· 5

L

Layer ····· 6
Layout ····· 26
LegendLayer ····· 7
LegendStyle ····· 20

M

Marker ····· 20
MarkerLayer ····· 7

O

Obstacles ····· 16

P

Palette ····· 20
Parameter ····· 24
PathLayer ····· 7
place_beside ····· 18
place_callout ····· 17
place_point_label ····· 18
Placement ····· 16
point_in_polygon ····· 13
PointLabelLayer ····· 15
polygon_edges ····· 12
polylabel ····· 13

Q

quadrant_axes ····· 8

R

ray_exit ····· 13
Rect ····· 11
rect_hits_segment ····· 12
rect_inside_polygon ····· 13
rect_overlaps_polygon ····· 13
RegionLabelLayer ····· 15
register_builder ····· 27
register_pass ····· 27
RenderCache ····· 28
Renderer ····· 27
RendererRegistry ····· 27
RolePack ····· 23

S

SaveOptions ····· 28
Scene ····· 6
segments_intersect ····· 12
Span ····· 26
SpanBraceLayer ····· 9
SparseStyle ····· 19
spread ····· 10
Stroke ····· 19
StyleBundle ····· 21

T

TextLayer ····· 7
TextStyle ····· 20
Theme ····· 21
ThemeRegistry ····· 23
TRANSPARENT ····· 20

U

use_config ····· 23

参考文献

Agafonkin, V. (2016,). *A New Algorithm for Finding a Visual Center of a Polygon*. https://blog.mapbox.com/a-new-algorithm-for-finding-a-visual-center-of-a-polygon-7c77e6492fbc

Ayer, M., Brunk, H. D., Ewing, G. M., Reid, W. T., & Silverman, E. (1955). An Empirical Distribution Function for Sampling with Incomplete Information. *The Annals of Mathematical Statistics*, *26*(4), 641–647. https://doi.org/10.1214/aoms/1177728423

Barlow, R. E., Bartholomew, D. J., Bremner, J. M., & Brunk, H. D. (1972). *Statistical Inference under Order Restrictions*. Wiley.

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

de Berg, M., Cheong, O., Kreveld, M. van, & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd ed.). Springer. https://doi.org/10.1007/978-3-540-77974-2

Haines, E. (1994). Point in Polygon Strategies. In P. S. Heckbert (Ed.), *Graphics Gems IV: Graphics Gems IV* (pp. 24–46). Academic Press.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55